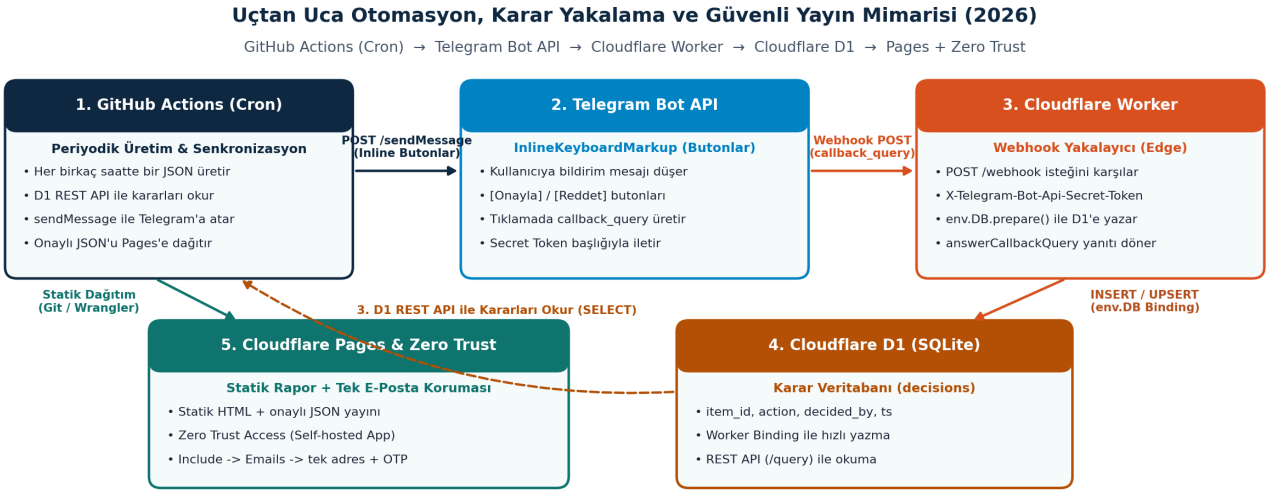
2026 RESMÎ DOKÜMANTASYON TABANLI UÇTAN UCA SİSTEM MİMARİSİ VE UYGULAMA KILAVUZU

GitHub Actions, Telegram Bot API ve Cloudflare (Workers, D1, Pages, Zero Trust) Kurulum Rehberi

Periyodik JSON Üretimi, Inline Butonlu Telegram Bildirimi, Webhook ile D1 Karar Kaydı, GitHub Actions Karar Okuma, Zero Trust Tek E-Posta Koruması, Sır Yönetimi ve 2026 Ücretsiz Katman Sınırları

Bu teknik kılavuz; birkaç saatte bir çalışan bir **GitHub Actions** iş akışının ürettiği JSON verilerini **Telegram Bot API** üzerinden etkileşimli (inline) butonlarla kullanıcıya sunmasını, kullanıcının buton tıklamalarını **Cloudflare Workers** webhook uç noktasıyla yakalayıp **Cloudflare D1** (sunucusuz SQL) veritabanına kaydetmesini, bir sonraki **GitHub Actions** çalışmasında bu kararların D1'den okunarak işlenmesini ve sonuçların **Cloudflare Pages** üzerinde yayınlanıp **Cloudflare Zero Trust Access** ile yalnızca tek bir yetkili e-posta adresine açılmasını adım adım açıklar.



Şekil 1: GitHub Actions, Telegram Bot API, Cloudflare Workers, D1 ve Pages + Zero Trust arasındaki çift yönlü veri ve onay akışı.

Bölüm	Kapsanan Teknik Adımlar ve Komutlar	Dayanılan Resmî Belge Kaynağı
Adım 1	`@BotFather` kurulumu, `getUpdates` ile `chat_id` alma, `sendMessage` ve `InlineKeyboardMarkup`	Telegram Bot API Docs & Bot Features
Adım 2	Wrangler ile D1 oluşturma, `wrangler.jsonc`, Worker `callback_query` işleme, `setWebhook` ve Secret Token	Cloudflare D1 Docs & Workers Docs
Adım 3	Periyodik `schedule` (cron) iş akışı, Cloudflare D1 HTTP REST API (`/query`) üzerinden kararları okuma	D1 REST API & GitHub Actions Docs
Adım 4	Cloudflare Pages statik site dağıtımı, Zero Trust Access Self-Hosted uygulama ve tek e-posta (One-Time PIN) kuralı	Cloudflare Pages & Zero Trust Access
Adım 5	`wrangler secret put`, GitHub Actions Encrypted Secrets ve Public depoda asla bulunmaması gereken veriler	Workers Secrets & GitHub Secrets
Adım 6	2026 güncel ücretsiz katman (Free Tier) kotaları, 1 Eylül 2026 D1 sert limit davranışı ve kota koruma önlemleri	D1 Limits, Workers Limits, Pages Limits

Belge Sürümü ve Doğrulama Tarihi: Ekim 2026 • Tüm uç noktalar, Wrangler v4 (`wrangler.jsonc`) yapılandırmaları ve Cloudflare D1/Workers/Pages/Zero Trust limitleri Ekim 2026 itibarıyla yürürlükte olan resmî geliştirici belgelerine göre doğrulanmıştır.

1. Adım 1: Telegram Botu Oluşturma, chat_id Alma ve Inline Butonlu Mesaj Gönderimi

Senaryomuzun ilk halkasında, GitHub Actions birkaç saatte bir JSON veri paketi ürettiğinde (örneğin yeni tespit edilen kayıtlar, fiyat/stok değişimleri veya onay bekleyen içerikler), bu öğeleri kullanıcının cep telefonuna veya masaüstü Telegram istemcisine **Onayla** ve **Reddet** butonlarıyla iletir. Telegram üzerinde bot oluşturma ve mesajlaşma işlemleri Telegram BotFather Resmî Kılavuzu ile Telegram Bot API Resmî Referansı kurallarına dayanır.

1.1. @BotFather Ekranı Üzerinden Bot Oluşturma ve Token Alma

Telegram uygulamasında arama çubuğuna `@BotFather` yazarak mavi onay rozetli resmî botu açın ve aşağıdaki adımları uygulayın:

1. **Komut:** Sohbet ekranına `/newbot` yazıp gönderin.
2. **Görünen Ad (Display Name):** BotFather sizden bir isim istediğinde örneğin `Karar Onay Otomasyon Botu` yazın.
3. **Kullanıcı Adı (Username):** Sonu `bot` ile biten benzersiz bir kullanıcı adı girin (ör. `ornek_karar_2026_bot`).
4. **Çıktı ve Bot Token:** BotFather size `1234567890:AAHxyz...` biçiminde bir HTTP API anahtarı (`TELEGRAM_BOT_TOKEN`) verir. Bu değeri güvenli bir yere not edin; **asla** kod deposuna yazmayın.

Önemli Teknik Ayrım: Mesajların altına eklenen tıklanabilir butonlar (`InlineKeyboardMarkup`) için BotFather'da ek bir ayar açmanıza gerek yoktur. BotFather'daki `/setinline` komutu yalnızca kullanıcıların herhangi bir sohbette `@bot_adi sorgu` yazarak satır içi arama yapmasını (Inline Mode) açar (Telegram Inline Keyboards Docs).

1.2. getUpdates Metodu ile Kişisel chat_id Değerini Öğrenme

Botlar güvenlik gereği kendilerine ilk mesajı atmayan bir kullanıcıya doğrudan mesaj gönderemez. Kendi `chat_id` numaranızı almak için:

1. Telegram'da yeni oluşturduğunuz botu açın, **Başlat (Start)** düğmesine basın ve `Merhaba` yazıp gönderin.
2. Henüz bir webhook tanımlı değilken terminalinizden Telegram getUpdates API uç noktasını çağırın (not: aktif bir webhook varken `getUpdates` çalışmaz ve 409 Conflict döner):

```
export TELEGRAM_BOT_TOKEN="1234567890:AAH_SIZIN_GIZLI_TOKENINIZ"

curl -s "https://api.telegram.org/bot${TELEGRAM_BOT_TOKEN}/getUpdates" | jq .
```

Dönen JSON yanıtı içindeki `result[0].message.chat.id` alanı sizin benzersiz `chat_id` değerinizdir:

```
{
  "ok": true,
  "result": [
    {
      "update_id": 849102341,
      "message": {
        "message_id": 1,
        "from": { "id": 987654321, "first_name": "Ahmet" },
        "chat": { "id": 987654321, "type": "private" },
        "date": 1793011200,
        "text": "Merhaba"
      }
    }
  ]
}
```

Buradaki `987654321` sayısını `TELEGRAM_CHAT_ID` olarak kaydedin.

1.3. sendMessage ve InlineKeyboardMarkup ile Butonlu Mesaj Gönderimi

GitHub Actions her çalıştığında ürettiği JSON içindeki her bir aday kayıt (ör. `item_2026_10_26_01`) için sendMessage uç noktasına `reply_markup` parametresiyle bir InlineKeyboardMarkup nesnesi gönderir. Her bir InlineKeyboardButton içindeki `callback_data` alanı **en fazla 1–64 bayt** uzunluğunda olabilir. Bu nedenle `callback_data` içine tüm JSON'u değil, `approve:item_2026_10_26_01` gibi kısa bir eylem ve kayıt kimliği çifti yerleştirilir:

```
curl -s -X POST "https://api.telegram.org/bot${TELEGRAM_BOT_TOKEN}/sendMessage"   -H "Content-Type:
application/json"   -d '{
    "chat_id": 987654321,
    "text": "Yeni JSON Kaydı Üretildi

Kayıt ID: item_2026_10_26_01
Başlık: Sunucu Kapasite Artırım Önerisi
Maliyet: 14.50 USD/ay

Lütfen kararınızı seçin:",
    "reply_markup": {
      "inline_keyboard": [
        [
          { "text": "✅ Onayla", "callback_data": "approve:item_2026_10_26_01" },
          { "text": "❌ Reddet", "callback_data": "reject:item_2026_10_26_01" }
        ]
      ]
    }
  }'
```

Resmî Belge Bağlantıları (Adım 1):

- BotFather ve Bot Özellikleri: https://core.telegram.org/bots/features#botfather
- `getUpdates` Metodu: https://core.telegram.org/bots/api#getupdates
- `sendMessage` Metodu: https://core.telegram.org/bots/api#sendmessage
- `InlineKeyboardMarkup` ve `InlineKeyboardButton` (64 bayt sınırı): https://core.telegram.org/bots/api#inlinekeyboardmarkup

---

2. Adım 2: Buton Tıklamasını Cloudflare Worker Webhook'u ile Yakalayıp D1'e Yazma

Kullanıcı Telegram'da **Onayla** veya **Reddet** butonuna tıkladığı anda Telegram sunucuları, önceden kaydettiğiniz HTTPS adresine bir `Update` JSON paketi (içinde `callback_query` barındıran POST isteği) gönderir (Telegram CallbackQuery Docs). Bu isteği 7/24 sunucu kiralamadan karşılamak için **Cloudflare Workers** ve kararları kalıcı SQL tablosunda tutmak için **Cloudflare D1** kullanacağız (Cloudflare D1 Getting Started).

2.1. Wrangler CLI Kurulumu, D1 Veritabanı ve Şema Oluşturma

Bilgisayarınızda Node.js kurulu iken Cloudflare'in resmî CLI aracı olan Wrangler'ı kullanarak projeyi ve D1 veritabanını oluşturun (Wrangler Install Docs):

```
# 1. Yeni bir Cloudflare Worker projesi oluşturun
npm create cloudflare@latest tg-d1-webhook -- --type=hello-world --ts=false --git=false
cd tg-d1-webhook

# 2. Cloudflare hesabınıza giriş yapın (tarayıcıda OAuth ekranı açılır)
npx wrangler login

# 3. D1 veritabanını oluşturun
npx wrangler d1 create kararlar-db
```

`wrangler d1 create kararlar-db` komutu terminalde size bir `database_id` (UUID) döndürür. Bu değeri `wrangler.jsonc` dosyanıza aşağıdaki gibi bağlayın (D1 Workers Binding API Docs):

```
{
  "$schema": "node_modules/wrangler/config-schema.json",
  "name": "tg-d1-webhook",
  "main": "src/index.js",
  "compatibility_date": "2026-10-01",
  "d1_databases": [
    {
      "binding": "DB",
      "database_name": "kararlar-db",
      "database_id": "BURAYA-WRANGLER-CIKTI-UUID-YAZIN"
    }
```

```
  ]
}
```

Şimdi `schema.sql` dosyasını oluşturup uzak (remote) D1 veritabanında kararlar tablosunu ve sorgu maliyetini düşüren indeksleri tanımlayın:

```
-- schema.sql
CREATE TABLE IF NOT EXISTS decisions (
  item_id TEXT PRIMARY KEY,
  action TEXT NOT NULL CHECK (action IN ('approve', 'reject')),
  telegram_user_id INTEGER NOT NULL,
  username TEXT,
  message_id INTEGER,
  decided_at TEXT NOT NULL,
  processed_by_gh INTEGER NOT NULL DEFAULT 0
);

CREATE INDEX IF NOT EXISTS idx_decisions_processed ON decisions(processed_by_gh, decided_at);
```

```
# Şemayı Cloudflare üzerindeki canlı (remote) D1 veritabanına uygulayın
npx wrangler d1 execute kararlar-db --remote --file=./schema.sql
```

2.2. Cloudflare Worker Kodu (src/index.js) — Doğrulama, D1 Yazma ve Mesaj Güncelleme

Worker kodumuz dört kritik güvenlik ve kullanıcı deneyimi adımını yerine getirir:

1. Telegram'ın gönderdiği `X-Telegram-Bot-Api-Secret-Token` HTTP başlığını kontrol ederek sahte webhook isteklerini reddeder (Telegram setWebhook secret_token).
2. Tıklayan kullanıcının `from.id` değerinin sizin yetkili `ALLOWED_CHAT_ID` değeriniz olup olmadığını doğrular.
3. Parametreli SQL (`env.DB.prepare(...).bind(...)`) kullanarak SQL enjeksiyonunu önler ve kararı D1'e yazar (D1 Prepared Statements).
4. answerCallbackQuery çağrısıyla Telegram'daki dönen yükleme animasyonunu durdurur ve editMessageText ile mesajın altındaki butonları kaldırarak verilen kararı metne yazar.

```
// src/index.js
export default {
  async fetch(request, env) {
    const url = new URL(request.url);
    if (request.method !== "POST" || url.pathname !== "/webhook") {
      return new Response("Not Found", { status: 404 });
    }

    // 1. Telegram Webhook Secret Token doğrulaması
    const headerSecret = request.headers.get("X-Telegram-Bot-Api-Secret-Token");
    if (!env.TELEGRAM_WEBHOOK_SECRET || headerSecret !== env.TELEGRAM_WEBHOOK_SECRET) {
      return new Response("Unauthorized webhook signature", { status: 401 });
    }

    const update = await request.json();
    if (!update.callback_query) {
      return new Response("OK", { status: 200 });
    }

    const cb = update.callback_query;
    const callerId = String(cb.from?.id || "");
    if (env.ALLOWED_CHAT_ID && callerId !== String(env.ALLOWED_CHAT_ID)) {
      return new Response("Forbidden user", { status: 403 });
    }

    // 2. callback_data ayrıştırma ("approve:item_2026_10_26_01")
    const rawData = cb.data || "";
    const [action, itemId] = rawData.split(":");
    if (!itemId || (action !== "approve" && action !== "reject")) {
      return new Response("Invalid callback_data", { status: 400 });
    }
```

```
    const nowIso = new Date().toISOString();
    const username = cb.from?.username || cb.from?.first_name || "unknown";
    const messageId = cb.message?.message_id || 0;

    // 3. D1 Veritabanına UPSERT (parametreli sorgu)
    await env.DB.prepare(
      `INSERT INTO decisions (item_id, action, telegram_user_id, username, message_id, decided_at,
processed_by_gh)
       VALUES (?1, ?2, ?3, ?4, ?5, ?6, 0)
       ON CONFLICT(item_id) DO UPDATE SET
         action = excluded.action,
         decided_at = excluded.decided_at,
         processed_by_gh = 0`
    )
      .bind(itemId, action, Number(callerId), username, messageId, nowIso)
      .run();

    const tgBase = `https://api.telegram.org/bot${env.TELEGRAM_BOT_TOKEN}`;
    const label = action === "approve" ? "✅ ONAYLANDI" : "❌ REDDEDİLDİ";

    // 4. Yükleme ikonunu kapat (answerCallbackQuery) ve mesajı güncelle (editMessageText)
    await fetch(`${tgBase}/answerCallbackQuery`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        callback_query_id: cb.id,
        text: `Karar kaydedildi: ${label} (${itemId})`
      })
    });

    if (cb.message?.chat?.id && cb.message?.message_id) {
      const originalText = cb.message.text || "";
      await fetch(`${tgBase}/editMessageText`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          chat_id: cb.message.chat.id,
          message_id: cb.message.message_id,
          text: `${originalText}

Sonuç: ${label} (${nowIso})`,
          reply_markup: { inline_keyboard: [] }
        })
      });
    }

    return new Response("OK", { status: 200 });
  }
};
```

2.3. Sırları Yükleme, Worker'ı Yayınlama ve Telegram setWebhook Kaydı

Sırları Wrangler ile şifreli olarak Worker'a yükleyin, Worker'ı dağıtın ve Telegram'a webhook adresinizi bildirin:

```
# 1. Worker sırlarını güvenli biçimde tanımlayın (komut sizden değeri gizli girmenizi ister)
npx wrangler secret put TELEGRAM_BOT_TOKEN
npx wrangler secret put TELEGRAM_WEBHOOK_SECRET
npx wrangler secret put ALLOWED_CHAT_ID

# 2. Worker'ı Cloudflare Edge ağına dağıtın
npx wrangler deploy

# 3. Telegram Bot API'ye Webhook URL ve Secret Token değerini kaydedin
export WORKER_URL="https://tg-d1-webhook.SIZIN-ALT-ALAN-ADINIZ.workers.dev/webhook"
export WEBHOOK_SECRET="Guclu_Rastgele_Webhook_Sirri_2026"

curl -s -X POST "https://api.telegram.org/bot${TELEGRAM_BOT_TOKEN}/setWebhook"   -H "Content-Type:
application/json"   -d "{
    "url": "${WORKER_URL}",
```

```
    "secret_token": "${WEBHOOK_SECRET}",
    "allowed_updates": ["callback_query"]
  }" | jq .

# 4. Webhook durumunu doğrulayın
curl -s "https://api.telegram.org/bot${TELEGRAM_BOT_TOKEN}/getWebhookInfo" | jq .
```

Resmî Belge Bağlantıları (Adım 2):

- Cloudflare D1 Başlangıç ve Şema Yürütme: https://developers.cloudflare.com/d1/get-started/
- Cloudflare D1 Workers Binding API (Prepared Statements): https://developers.cloudflare.com/d1/workers-binding-api/
- Telegram `setWebhook` ve `getWebhookInfo`: https://core.telegram.org/bots/api#setwebhook
- Telegram `CallbackQuery` ve `answerCallbackQuery`: https://core.telegram.org/bots/api#callbackquery

---

3. Adım 3: GitHub Actions İş Akışının D1'den Kararları Okuması ve Yeni JSON Üretmesi

GitHub Actions, Cloudflare Worker ortamının dışında (GitHub sunucularında) çalıştığı için D1 veritabanına `env.DB` binding ile değil, **Cloudflare D1 HTTP REST API** uç noktası üzerinden erişir (Cloudflare D1 REST API Query Docs ve Query D1 Best Practices).

3.1. En Az Yetkili (Least-Privilege) Cloudflare API Token Oluşturma

1. **Ekran:** Cloudflare Dashboard → sağ üst profil simgesi → **Profile** → **API Tokens** → **Create Token** → **Create Custom Token** düğmesine tıklayın (Create API Token Docs).
2. **İzinler (Permissions):** `Account` → `D1` → `Edit` (veya yalnızca `SELECT` yapacaksanız `Read`; ancak işlenen kayıtları `processed_by_gh = 1` olarak işaretlemek ve Pages'e dağıtım yapmak için `Account - D1 - Edit` ve `Account - Cloudflare Pages - Edit` izinlerini ekleyin).
3. **Account Resources:** `Include - Specific account` seçerek yalnızca kendi hesabınızı kapsam içine alın ve **Continue to summary** → **Create Token** ile üretilen değeri `CF_API_TOKEN` olarak GitHub Secrets'a kaydedin.

3.2. Cloudflare D1 REST API Sorgu Formatı

D1 REST API sorgu uç noktası `POST` `https://api.cloudflare.com/client/v4/accounts/{account_id}/d1/database/{database_id}/query` adresidir ve parametreli SQL için `sql` ile `params` dizisini kabul eder:

```
curl -s -X POST
"https://api.cloudflare.com/client/v4/accounts/${CF_ACCOUNT_ID}/d1/database/${CF_D1_DATABASE_ID}/query"
-H "Authorization: Bearer ${CF_API_TOKEN}"   -H "Content-Type: application/json"   -d '{
    "sql": "SELECT item_id, action, decided_at FROM decisions WHERE processed_by_gh = ?;",
    "params": [0]
  }' | jq .
```

3.3. GitHub Actions İş Akışı (.github/workflows/pipeline.yml) ve Python Betiği

Aşağıdaki iş akışı her 4 saatte bir (`0 */4 * * *`) veya manuel olarak (`workflow_dispatch`) çalışır; önce D1'deki kararları okur, onaylananları statik site için `dist/approved.json` dosyasına yazar, ardından yeni üretilen aday kayıtları Telegram'a inline butonlu mesaj olarak gönderir (GitHub Actions Schedule Docs):

```
# .github/workflows/pipeline.yml
name: Periyodik JSON Uretimi ve D1 Karar Senkronizasyonu

on:
  schedule:
    - cron: '0 */4 * * *'   # Her 4 saatte bir çalışır (POSIX cron)
  workflow_dispatch:        # Manuel tetikleme imkanı

permissions:
  contents: read

jobs:
```

```
  sync-and-publish:
    runs-on: ubuntu-latest
    steps:
      - name: Depoyu indir
        uses: actions/checkout@v4

      - name: Python 3.12 kur
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: D1 kararlarini oku, yeni JSON uret ve Telegram bildirimi gonder
        env:
          CF_ACCOUNT_ID: ${{ secrets.CF_ACCOUNT_ID }}
          CF_D1_DATABASE_ID: ${{ secrets.CF_D1_DATABASE_ID }}
          CF_API_TOKEN: ${{ secrets.CF_API_TOKEN }}
          TELEGRAM_BOT_TOKEN: ${{ secrets.TELEGRAM_BOT_TOKEN }}
          TELEGRAM_CHAT_ID: ${{ secrets.TELEGRAM_CHAT_ID }}
        run: python3 scripts/sync_and_notify.py

      - name: Statik siteyi Cloudflare Pages uzerine dagit
        uses: cloudflare/wrangler-action@v3
        with:
          apiToken: ${{ secrets.CF_API_TOKEN }}
          accountId: ${{ secrets.CF_ACCOUNT_ID }}
          command: pages deploy dist --project-name=ozel-karar-paneli
```

`scripts/sync_and_notify.py` betiğinin D1'den kararları okuyan ve yeni JSON öğelerini Telegram'a gönderen temel yapısı:

```
# scripts/sync_and_notify.py
import json, os, urllib.request
from datetime import datetime, timezone

CF_ACCOUNT_ID = os.environ["CF_ACCOUNT_ID"]
CF_DB_ID = os.environ["CF_D1_DATABASE_ID"]
CF_TOKEN = os.environ["CF_API_TOKEN"]
TG_TOKEN = os.environ["TELEGRAM_BOT_TOKEN"]
TG_CHAT_ID = os.environ["TELEGRAM_CHAT_ID"]

def query_d1(sql, params=None):
    url = f"https://api.cloudflare.com/client/v4/accounts/{CF_ACCOUNT_ID}/d1/database/{CF_DB_ID}/query"
    payload = json.dumps({"sql": sql, "params": params or []}).encode("utf-8")
    req = urllib.request.Request(url, data=payload, method="POST", headers={
        "Authorization": f"Bearer {CF_TOKEN}",
        "Content-Type": "application/json"
    })
    with urllib.request.urlopen(req, timeout=20) as resp:
        data = json.loads(resp.read().decode("utf-8"))
        return data["result"][0]["results"]

# 1. D1'den onaylanmış tüm kararları oku ve statik site için kaydet
approved_rows = query_d1(
    "SELECT item_id, action, username, decided_at FROM decisions WHERE action = ? ORDER BY decided_at
DESC LIMIT 100;",
    ["approve"]
)
os.makedirs("dist", exist_ok=True)
with open("dist/approved.json", "w", encoding="utf-8") as f:
    json.dump({"updated_at": datetime.now(timezone.utc).isoformat(), "items": approved_rows}, f,
ensure_ascii=False, indent=2)

# 2. Yeni üretilen JSON öğesini Telegram'a inline butonlu olarak gönder
new_item_id = datetime.now(timezone.utc).strftime("item_%Y%m%d_%H%M")
tg_url = f"https://api.telegram.org/bot{TG_TOKEN}/sendMessage"
tg_payload = json.dumps({
    "chat_id": int(TG_CHAT_ID),
    "text": f"Yeni Otomatik Rapor Kaydı
```

```
ID: {new_item_id}
Oluşturulma: {datetime.now(timezone.utc).isoformat()}",
    "reply_markup": {
        "inline_keyboard": [[
            {"text": "✅ Onayla", "callback_data": f"approve:{new_item_id}"},
            {"text": "❌ Reddet", "callback_data": f"reject:{new_item_id}"}
        ]]
    }
}).encode("utf-8")
urllib.request.urlopen(urllib.request.Request(tg_url, data=tg_payload, headers={"Content-Type":
"application/json"}))
```

Resmî Belge Bağlantıları (Adım 3):

- Cloudflare D1 REST API Query Uç Noktası: https://developers.cloudflare.com/api/resources/d1/subresources/database/methods/query/
- D1 Sorgulama En İyi Uygulamaları: https://developers.cloudflare.com/d1/best-practices/query-d1/
- Cloudflare GitHub Actions Entegrasyonu: https://developers.cloudflare.com/workers/ci-cd/external-cicd/github-actions/
- GitHub Actions `schedule` (Cron) Tetikleyicisi: https://docs.github.com/en/actions/writing-workflows/choosing-when-your-workflow-runs/events-that-trigger-workflows#schedule

---

4. Adım 4: Statik Siteyi Cloudflare Pages'e Koyup Zero Trust ile Yalnız Tek E-Postaya Açma

`dist/index.html` ve `dist/approved.json` dosyalarından oluşan statik sitenizi **Cloudflare Pages** üzerinde yayınladıktan sonra, sitenin internetteki herkes tarafından görüntülenmesini engellemek ve yalnızca sizin tek bir e-posta adresinize izin vermek için **Cloudflare Zero Trust (Cloudflare One) Access** katmanını devreye alacağız (Cloudflare Pages Access Integration Docs ve Cloudflare Access Policies Docs).

4.1. Cloudflare Pages Projesi Oluşturma ve Statik Site Dağıtımı

1. **CLI ile İlk Proje Oluşturma:** Terminalde `npx wrangler pages project create ozel-karar-paneli --production-branch=main` komutunu çalıştırın veya **Cloudflare Dashboard → Compute (Workers & Pages) → Create → Pages** ekranından oluşturun (Pages Direct Upload Docs).
2. **Özel Alan Adı (Custom Domain) Bağlama:** Pages projenizin içinde **Custom domains → Set up a custom domain** yolunu izleyerek örneğin `panel.sizinalanadiniz.com` adresini ekleyin (Pages Custom Domains Docs).
3. **Önizleme (*.pages.dev) URL'lerini Kapatma:** Ziyaretçilerin özel alan adındaki Zero Trust duvarını atlayıp doğrudan `ozel-karar-paneli.pages.dev` adresine girmesini önlemek için **Workers & Pages → ozel-karar-paneli → Settings → General → Access Policy** altında **Enable Access policy** düğmesine tıklayarak `*.ozel-karar-paneli.pages.dev` önizleme adreslerini de Access korumasına alın.

4.2. Cloudflare Zero Trust Access ile Tek E-Posta Kısıtlaması (Adım Adım Ekranlar)

1. **Zero Trust Paneline Giriş:** https://one.dash.cloudflare.com/ adresinden Cloudflare Zero Trust (Cloudflare One) kontrol panelini açın.
2. **Kimlik Sağlayıcı (One-Time PIN) Kontrolü:** Sol menüden **Settings → Authentication → Login methods** ekranına gidin ve **One-time PIN (OTP)** yönteminin aktif olduğundan emin olun. Bu yöntem, harici bir OAuth uygulaması kurmadan izin verilen e-posta adresine 6 haneli tek kullanımlık giriş kodu gönderir (One-Time PIN Docs).
3. **Uygulama Ekleme:** Sol menüden **Access → Applications → Add an application → Self-hosted** seçeneğine tıklayın (Self-Hosted Apps Docs).
4. **Alan Adı Tanımı:** *Application name* alanına `Karar Paneli` yazın, *Session Duration* süresini belirleyin (ör. `24 hours`) ve **Add public hostname** bölümüne hem `panel.sizinalanadiniz.com` hem de `ozel-karar-paneli.pages.dev` adreslerini ekleyin.
5. **Tek E-Posta Politikası (Access Policy):**
   - **Policy name:** `Yalnizca Tek Yetkili E-Posta`

- **Action:** `Allow`
- **Include (Kapsama Kuralı):** *Selector* açılır menüsünden `Emails` seçin ve *Value* kutusuna **yalnızca kendi e-posta adresinizi** (ör. `adiniz@ornek.com`) yazın.

6. **Kaydetme ve Test: Next → Add application** ile kurulumu tamamlayın. Artık sitenize giren herkes Cloudflare Access giriş ekranıyla karşılaşır; yalnızca `adiniz@ornek.com` adresine tek kullanımlık PIN kodu gönderilir ve diğer tüm e-posta adresleri reddedilir.

Resmî Belge Bağlantıları (Adım 4):

- Cloudflare Pages Özel Alan Adları ve Access Koruması: https://developers.cloudflare.com/pages/configuration/custom-domains/
- Cloudflare Zero Trust Access Politikaları: https://developers.cloudflare.com/cloudflare-one/access-controls/policies/
- Zero Trust One-Time PIN (E-Posta OTP) Doğrulaması: https://developers.cloudflare.com/cloudflare-one/identity/one-time-pin/

---

5. Adım 5: Sırların (Secrets) Nerede Tutulacağı ve Public GitHub Deposunda Asla Bulunmaması Gerekenler

Özellikle **Public (herkese açık)** bir GitHub deposu kullanırken tek bir commit içinde bile unutulan bir API anahtarı veya bot token'ı, saniyeler içinde otomatik tarama botları tarafından ele geçirilebilir. Sırların yaşam döngüsü Cloudflare Workers Secrets Docs ve GitHub Actions Security Guides kurallarına göre ayrıştırılmalıdır.

5.1. Hangi Sır Nerede Tutulmalı? (Sır Envanter Tablosu)

Gizli Değer / Değişken	Tutulacağı Güvenli Ortam	Tanımlama Komutu / Ekran Yolu ve Kullanım Amacı
`TELEGRAM_BOT_TOKEN`	**Hem** GitHub Actions Secrets **hem** Cloudflare Worker Secrets	GitHub: *Settings → Secrets and variables → Actions* Worker: `npx wrangler secret put TELEGRAM_BOT_TOKEN` Mesaj gönderme ve callback yanıtı için kullanılır.
`TELEGRAM_WEBHOOK_SECRET`	Cloudflare Worker Secrets	`npx wrangler secret put TELEGRAM_WEBHOOK_SECRET` Yalnızca Telegram'dan gelen `X-Telegram-Bot-Api-Secret-Token` başlığını doğrulamak için kullanılır.
`CF_API_TOKEN`	GitHub Actions Repository Secrets	*Settings → Secrets and variables → Actions → New repository secret* GitHub Actions'ın D1 REST API sorgusu ve Pages dağıtımı yapması için kullanılır.
`CF_ACCOUNT_ID` ve `CF_D1_DATABASE_ID`	GitHub Actions Secrets (veya Variables)	Public depoda hesap ve veritabanı topolojisini ifşa etmemek için GitHub Secrets altında tutulması önerilir.
`TELEGRAM_CHAT_ID` / `ALLOWED_CHAT_ID`	GitHub Actions Secrets & Cloudflare Worker Secrets	Kişisel Telegram kullanıcı kimliğinizin public depoda görünmemesi ve spam almamanız için Secret olarak saklanır.
Yerel Geliştirme Sırları	`.dev.vars` (Yalnızca yerel diskte, `.gitignore` içinde)	`wrangler dev` çalışırken yerel ortamda okunur; **asla** Git'e eklenmez (Local Dev Secrets Docs).

5.2. Public GitHub Deposunda ASLA Bulunmaması Gerekenler

- **Ham Token ve Anahtarlar:** `TELEGRAM_BOT_TOKEN`, `CF_API_TOKEN`, Cloudflare Global API Key ve `TELEGRAM_WEBHOOK_SECRET` kaynak kodda, `wrangler.jsonc` içinde (`"vars": {...}` altında) veya YAML dosyalarında

düz metin olarak **asla** yer almamalıdır.

- **Yerel Ortam ve Veritabanı Dosyaları:** `.env`, `.dev.vars`, `.wrangler/state/` (yerel SQLite D1 kopyaları) ve ham müşteri/kullanıcı e-posta adresleri içeren SQL dökümleri (`dump.sql`) depoya gönderilmemelidir.
- **İstemci Tarafı (Frontend) JavaScript İçinde API Çağrısı:** Cloudflare Pages'e yüklenen statik HTML/JS dosyaları tarayıcıda çalıştığı için içine `CF_API_TOKEN` veya `TELEGRAM_BOT_TOKEN` gömülmemelidir; tüm D1 okumaları GitHub Actions derleme aşamasında veya sunucu tarafı Worker/Pages Functions içinde yapılmalıdır.
- **GitHub Actions Loglarına Sır Yazdırma:** İş akışı içinde `echo ${{ secrets.CF_API_TOKEN }}` veya `curl -v` ile Authorization başlığını loglara döken hata ayıklama komutları kullanılmamalıdır. Ayrıca `pull_request_target` tetikleyicisi dışarıdan gelen PR'lara sırları açabileceği için public depolarda kullanılmamalıdır.

Depo kök dizininde bulunması zorunlu `.gitignore` şablonu ve GitHub Secret Scanning koruması (GitHub Secret Scanning Docs):

```
# .gitignore — Public GitHub deposu için zorunlu dışlamalar
.env
.env.*
.dev.vars
.dev.vars.*
.wrangler/
node_modules/
dist/
*.sqlite
*.db
```

Ek olarak GitHub deponuzda **Settings → Code security → Secret scanning** ve **Push protection** özelliklerini etkinleştirin; böylece yanlışlıkla bir token commit etmeye çalıştığınızda GitHub `git push` işlemini anında reddeder.

Resmî Belge Bağlantıları (Adım 5):

- Cloudflare Workers Sır Yönetimi (`wrangler secret` ve `.dev.vars`): https://developers.cloudflare.com/workers/configuration/secrets/
- GitHub Actions içinde Sırların Güvenli Kullanımı: https://docs.github.com/en/actions/security-for-github-actions/security-guides/using-secrets-in-github-actions
- GitHub Secret Scanning ve Push Protection: https://docs.github.com/en/code-security/secret-scanning/about-secret-scanning

---

6. Adım 6: 2026 Güncel Ücretsiz Katman (Free Tier) Sınırları ve Kota Yönetimi

Bu mimarideki tüm bileşenler (Telegram Bot API, Cloudflare Workers, D1, Pages, Zero Trust Access ve GitHub Actions) kişisel ve küçük/orta ölçekli otomasyon senaryoları için tamamen ücretsiz katmanda (**0 USD/ay**) çalışacak şekilde tasarlanmıştır. Ancak özellikle **1 Eylül 2026** tarihinde yürürlüğe giren Cloudflare D1 günlük kota uygulaması (Cloudflare D1 Free Tier Limit Enforcement Changelog) gibi sınırların bilinmesi kesintisiz çalışma için kritiktir.

Hizmet / Bileşen	2026 Ücretsiz Katman (Free Tier) Sınırları	Aşım Durumu ve 2026 Kritik Notu	Resmî Belge
Cloudflare Workers (Free)	• **100.000 istek / gün** (00:00 UTC sıfırlama) • İstek başına **10 ms CPU süresi** (Ağ/D1 bekleme süresi CPU'ya sayılmaz) • İzolat başına **128 MB RAM** • İstek başına **50 subrequest** • Maksimum kod boyutu: **64 MiB** (Eylül 2026'da artırıldı) • Worker başına **64 ortam değişkeni/sır**	Günlük 100.000 istek aşılırsa `Error 1027` döner; 10 ms CPU aşılırsa `Error 1102` döner. Webhook senaryosunda günde birkaç yüz tıklama bu limitin %1'ini bile kullanmaz.	Workers Limits & Workers Pricing

Hizmet / Bileşen	2026 Ücretsiz Katman (Free Tier) Sınırları	Aşım Durumu ve 2026 Kritik Notu	Resmî Belge
Cloudflare D1 (Workers Free)	• Okunan satır (`rows_read`): **5 milyon satır / gün** • Yazılan satır (`rows_written`): **100.000 satır / gün** • Depolama: **Toplam 5 GB** (veritabanı başına maks. **500 MB**) • Hesap başına **10 veritabanı** • Worker çağrısı başına **50 sorgu** • Time Travel (geri yükleme): **7 gün**	**1 Eylül 2026 Değişikliği:** Günlük 5M okuma veya 100K yazma sınırı aşıldığında sorgular artık 00:00 UTC'ye kadar **hata vererek durur**. Tam tablo taraması (`SELECT *`) yerine mutlaka indeks (`idx_decisions_processed`) kullanın.	D1 Limits, D1 Pricing & Eylül 2026 Duyurusu
Cloudflare Pages (Free)	• Aylık derleme: **500 build / ay** • Eşzamanlı derleme: **1 build** (20 dk zaman aşımı) • Site başına **20.000 dosya**, dosya başına **25 MiB** • Proje başına **100 özel alan adı** • Statik bant genişliği: **Sınırsız (ücretsiz egress)**	GitHub Actions her 4 saatte bir (günde 6 kez = ayda ~180 kez) dağıtım yaptığında aylık 500 derleme limitinin oldukça altında kalır.	Pages Limits
Cloudflare Zero Trust Access (Free)	• Kullanıcı sayısı: **50 kullanıcıya (seat) kadar ücretsiz** • Uygulama: **50 özel uygulama** (Access politikalarında 500 uygulama sınırı) • Kural başına **1.000 e-posta adresi** • Denetim günlükleri (Audit logs): Ücretsiz planda **24 saat** saklama	Tek e-posta adresine açılan statik site yalnızca **1 kullanıcı koltuğu (seat)** tüketir; 50 kullanıcı sınırının çok altındadır.	Zero Trust Account Limits & Zero Trust Plans
Telegram Bot API	• Bot API kullanımı: **Tamamen ücretsiz** • Tekil sohbete mesaj hızı: **~1 mesaj / saniye** • Toplu mesaj hızı: **~30 mesaj / saniye** • `callback_data` boyutu: **1 – 64 bayt**	Hız sınırları aşılırsa HTTP 429 `Too Many Requests` ve `retry_after` saniyesi döner. Birkaç saatte bir gönderilen bildirimler hız sınırına takılmaz.	Telegram Bot FAQ Limits & Bot API
GitHub Actions (Free)	• **Public Depo:** Standart GitHub-hosted runner'larda **sınırsız ücretsiz dakika** • **Private Depo (Free):** Aylık **2.000 dakika / ay** ve 500 MB Artifacts depolama • Minimum `schedule` aralığı: **5 dakika**	60 gün boyunca hiçbir commit veya aktivite olmayan depolarda zamanlanmış (`schedule`) iş akışları GitHub tarafından otomatik duraklatılır; arayüzden yeniden etkinleştirilebilir.	GitHub Actions Billing & Actions Limits

6.1. Ücretsiz Katmanda Kalmak İçin 3 Altın Mühendislik Kuralı

- **D1 İndeks Zorunluluğu:** D1'de `rows_read` metriği dönen satır sayısını değil, *taranan (scanned)* satır sayısını ölçer. 10.000 satırlık bir tabloda indekssiz `SELECT * FROM decisions WHERE processed_by_gh = 0` çalıştırmak her seferinde 10.000 satır okuma harcar. Adım 2.1'de oluşturduğumuz `CREATE INDEX idx_decisions_processed ON decisions(processed_by_gh, decided_at)` indeksi sayesinde yalnızca eşleşen birkaç satır okunur.
- **Cloudflare Pages Build Kotasını Korumak:** GitHub Actions her çalıştığında yeni onaylanmış bir karar yoksa (D1'den 0 yeni satır döndüyse) `wrangler pages deploy` adımını koşullu (`if: steps.sync.outputs.has_changes == 'true'`) çalıştırarak aylık 500 derleme kotasını gereksiz yere harcamayın.
- **Telegram Webhook Filtrelemesi:** `setWebhook` çağrısında `"allowed_updates": ["callback_query"]` belirterek botun gereksiz grup mesajları veya durum güncellemeleri için Worker'ınızı tetiklemesini önleyin.